

Timers, Part 1: SysTick and the TIM Time Base

Replacing software delay loops with hardware that counts. A counter, a prescaler, and the arithmetic that turns clock ticks into seconds

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 7, printed pages 205–212 and 219–225.

Lab manual: Lab 04 (timers, delay and interrupt modes), Lab 05 (PWM setup), Lab 06 (capture/compare).

1 What we are actually after this week

Week 04 put `Delay()` on trial and convicted it. Week 05 gestured at a hardware timer as the fix. Week 11 needed a hardware trigger for the ADC and did not have one. This is the week all three debts get paid.

The question: **how do you make something happen at an exact time, repeatedly, without the CPU watching the clock?**

The plan:

1. The core idea, which is one counter and one clock, and the arithmetic that connects them to seconds.
2. What a timer peripheral is actually *for*. Five uses, and they are more varied than "delay".
3. **SysTick**: the small timer built into every Cortex-M core, and a worked 1 Hz example that runs into a real constraint.
4. **The TIM module**: the general-purpose timers, the time-base unit, and the **prescaler**.
5. **The frequency formula**, which is the calculation you will be asked to do.
6. Periodic interrupts from a TIM, with real measured overhead numbers.

Item 5 is the guaranteed exam calculation. Item 3 contains a constraint that teaches you why the prescaler in item 4 exists, so read them in order. Item 6 has the numbers that justify the whole chapter.

2 Deep dive 1: A counter and a clock

The core of every timer peripheral is embarrassingly simple.

Timer/counter

Counter: a digital circuit that counts the number of input pulses.

Timer/counter: a peripheral that measures time or counts events. It is the same circuit either way. The difference is entirely in what you feed it.

Feed it pulses from an **unknown** source and it counts *events*. Feed it pulses from a source of **known frequency** and it measures *time*.

That last sentence is the whole trick and it is worth sitting with. A counter has no concept of time. It knows only how many pulses have arrived. Time appears only because you happen to know how far apart the pulses are.

2.1 The arithmetic

If the input clock is 10 MHz, the period is $1/(10 \text{ MHz}) = 0.1 \mu\text{s}$. So one count represents $0.1 \mu\text{s}$ of elapsed time.

Read the counter and multiply. A count of 15821, up-counting, means

$$15821 \times 0.1 \mu\text{s} = 1582.1 \mu\text{s}$$

have passed since the counter was reset. That is the entire measurement principle.

Beware the trap

Notice what this buys you over the busy-wait from Week 04, and be precise about it, because it is a likely exam answer.

The delay loop's duration depended on the compiler, the optimisation level, the flash wait states, and whether an interrupt fired. A hardware counter's duration depends on *one* thing: the frequency of its input clock. It is unaffected by what the CPU is doing, by interrupts, by optimisation, by anything.

That is the difference between "roughly 400 units" and "exactly 1582.1 microseconds". And it is why every timing-critical thing in the rest of the course goes through a timer.

The surrounding circuitry adds flexibility: which signal source to count, which edge counts, whether to count up or down, what to do on overflow, and so on. But the counter is the thing.

3 Deep dive 2: What timers are actually used for

Five uses, and they are worth knowing as a set because it explains why the peripheral has as many registers as it does.

3.1 Periodic timer tick

Generate an interrupt at a fixed interval. The interval is steady **because it is controlled by hardware and not affected by software delays**, which is precisely the property Week 04 lacked.

This is also how you replace a blocking delay properly: rather than spinning, a periodic interrupt tracks elapsed time and starts the task when enough of it has passed. Recall the wait states from Week 05's FSM: this is what fills them.

3.2 Timestamps and range extension

The counter register measures the current time at high resolution, which is how you timestamp logged events.

But a 16-bit counter at, say, 1 MHz wraps every 65.5 ms, which is a short history. The fix is to **use the overflow interrupt to increment a software counter**, giving you a value as wide as you like. Recall Week 08: that software counter is shared between an ISR and main-line code, so it must be *volatile*, and reading it alongside the hardware counter is a multi-step access needing care.

3.3 Watchdog

A timer that resets the MCU if the software fails to service it. Recall the reliability attribute from Week 01. Covered properly next week.

3.4 Frequency and period measurement

The book's example is a cup anemometer: three arms with cups, magnets near the axle, and a reed switch on the mast that pulses each time a magnet passes. Pulse frequency is proportional to wind speed:

$$v_{wind} \approx 2\pi r f_{anem}$$

And there are **two ways** to measure that frequency, which is a genuinely useful distinction:

	Count events	Measure period
Mode	Event counter	Timer, with input capture
Method	Clear, run for a fixed time, read the count	Capture the counter on successive rising edges
Result	$f = \text{count}/\text{time}$	$f = 1/T$
Good for	High frequencies	Low frequencies

Which method, and why

Counting events over a fixed window gives you a count with ± 1 resolution, so at high frequency the count is large and the relative error is tiny. At low frequency you might count 3 pulses in a second, and ± 1 is a 33% error.

Measuring the period of a single cycle is the reverse: at low frequency the period is many timer counts and the relative error is tiny; at high frequency the period is a handful of counts and the error is large.

So: count events for fast signals, measure period for slow ones. Your Lab 06 does frequency measurement, and this is the answer to its evaluation question about what affects accuracy.

3.5 PWM generation

Covered next week, but the definition belongs here since it motivates the channel hardware:

PWM and duty cycle

Pulse-width modulation: a method for encoding information on a single digital signal line based on **duty cycle**, rather than encoding it serially as a sequence of bits.

Duty cycle: the fraction of time the signal is asserted.

Because it is a digital signal, it is **much less vulnerable to electrical noise** than an analog voltage carrying the same information.

And recall the hot plate from Week 01. PWM is exactly the fix for its binary output: instead of fully on or fully off, the heating element receives an average power proportional to duty cycle, giving precise control with less overshoot. The chapter says so explicitly, which is a nice piece of the book closing its own loop.

4 Deep dive 3: SysTick

SysTick

A simple timer peripheral contained **in the Cortex-M CPU core itself**, designed to provide a periodic tick for a scheduler or OS kernel.

Because it is part of the core rather than the vendor's peripheral set, **every Cortex-M processor has one, from every manufacturer, with the same registers**. That portability is its main advantage.

The counter is **24 bits** and **counts down**. Three registers, plus a calibration one:

Register	CMSIS	Contents
SYST_CVR	SysTick->VAL	CURRENT, the live count. Writing anything clears it to zero
SYST_RVR	SysTick->LOAD	RELOAD, the value loaded on wrap
SYST_CSR	SysTick->CTRL	CLKSOURCE, TICKINT, ENABLE, COUNTFLAG
SYST_CALIB	—	Calibration support

Operation: when enabled it loads CURRENT from RELOAD, counts down each clock pulse, and on reaching zero reloads and optionally raises a SysTick exception.

The control fields:

- CLKSOURCE: 1 selects the processor clock, 0 selects an external reference clock. On the book's part that reference is **the processor clock divided by 8**.
- TICKINT: 1 enables the exception on reaching zero.
- ENABLE: 1 starts the counter.
- COUNTFLAG: reads 1 if the timer has hit zero since this register was last read. **Reading SYST_CSR clears it**, as does writing SYST_CVR.

The off-by-one, and it is the exam's favourite

The counter divides the input frequency by RELOAD + 1, not by RELOAD.

Because the count sequence includes zero: from RELOAD down to 0 inclusive is RELOAD + 1 values.

So to divide f_{in} by N , store $N - 1$ in RELOAD. Write N and your period is one tick long. This same +1 appears in the TIM prescaler and auto-reload registers, so learn it once.

4.1 Worked example: a 1 Hz interrupt, and the constraint it hits

Processor clock is 48 MHz. We want 1 Hz.

$$N = \frac{48 \text{ MHz}}{1 \text{ Hz}} = 48,000,000 \Rightarrow \text{RELOAD} = 47,999,999$$

Now check it fits. The field is 24 bits, so the maximum is $2^{24} - 1 = 16,777,215$. And 47,999,999 is nearly three times that. **It does not fit.**

So use the alternate clock source, CLKSOURCE= 0, which runs at $48 \text{ MHz}/8 = 6 \text{ MHz}$:

$$N = \frac{6 \text{ MHz}}{1 \text{ Hz}} = 6,000,000 \Rightarrow \text{RELOAD} = 5,999,999$$

And $5,999,999 < 16,777,215$, so it fits.

Listing 1: Book Listing 7.1. Note the order of operations.

```

1 #define F_SYS_CLK (48000000L)
2
3 void Init_SysTick(void) {
4     /* 1. reload value for 1 Hz from the divided-by-8 clock */
5     MODIFY_FIELD(SysTick->LOAD, SysTick_LOAD_RELOAD, (F_SYS_CLK/8) - 1);
6     /* 2. interrupt priority */
7     NVIC_SetPriority(SysTick_IRQn, 3);
8     /* 3. force the reload value to load now */
9     MODIFY_FIELD(SysTick->VAL, SysTick_VAL_CURRENT, 0);
10    /* 4. enable interrupt and counter (CLKSOURCE left 0 = /8 clock) */
11    SysTick->CTRL = SysTick_CTRL_TICKINT_Msk | SysTick_CTRL_ENABLE_Msk;
12 }
13
14 void SysTick_Handler(void) {
15     static int n = 0;
16     Control_RGB_LEDs(n & 1, n & 2, n & 4);
17     n++;
18 }
19
20 int main(void) {
21     Set_Clocks_To_48MHz();
22     Init_GPIO_RGB();
23     Init_SysTick();
24     SystemCoreClockUpdate();
25     while (1)
26         ; /* the handler does all the work */
27 }

```

Look at that main. It is an empty infinite loop, and the entire behaviour of the program happens in an interrupt handler. Compare it against Week 04's starter program, which spent all its time inside Delay. Same visible result, and the CPU here is free to do essentially anything else.

Beware the trap

Why the range limit matters more than it looks.

SysTick's 24-bit counter with only two clock choices means the achievable periods form a fairly narrow band. At 48 MHz you can reach at most $16,777,216/48 \text{ MHz} \approx 350 \text{ ms}$, and with the $\div 8$ source at most about 2.8 s.

Want a 10-second tick? SysTick cannot do it directly. You set it to 1 s and count ten interrupts in software. Want 1 μs resolution *and* a 5-second period? You cannot have both, because that needs 24 bits of range at 1 MHz and you only have 16.7 bits' worth.

This limitation is exactly what the TIM module's prescaler exists to solve, and meeting the problem before the solution is the right order to learn it in.

In the lab

On your board, HAL_Init() configures SysTick for a 1 ms tick and SysTick_Handler calls HAL_IncTick(), which increments a millisecond counter. That is what HAL_Delay() counts, and it is why HAL_Delay(500) really is 500 ms while the book's Delay(500) is 500 of nothing in particular.

Two consequences worth knowing. First, **if you disable interrupts for a long time, HAL_Delay stalls**, because its tick stops arriving. Recall the Week 07 warning about critical section length; this is one of the ways it bites.

Second, **if you call HAL_Delay from inside an ISR whose priority is at least as urgent as SysTick's, it hangs forever**, since SysTick can never preempt to advance the counter. That is a classic first-time bug and the symptom is a board that freezes on its first button press.

5 Deep dive 4: The TIM module

SysTick is one counter with one job. The TIM peripherals are the general-purpose article.

Book board vs your board

Book, STM32F091RC: 10 TIMs. One 16-bit advanced-control timer (TIM1) with four channels, one **32-bit** timer (TIM2) with four channels, and seven 16-bit timers with up to four channels each (TIM3, TIM6, TIM7, TIM14, TIM15, TIM16, TIM17). The chapter uses **TIM3**.

Yours, STM32F303VCT6: a comparable but not identical set, including **two** advanced-control timers, **TIM1 and TIM8**, which the F091 does not have. Advanced-control timers matter because they provide **complementary outputs with dead-time insertion**, which is what you need to drive an H-bridge without shoot-through.

Not every timer has every feature. TIM6 and TIM7 are "basic" timers with a time base and no channels at all, so they can generate periodic interrupts and trigger the DAC or ADC, and cannot do PWM or input capture. Check the reference manual's timer feature table before assigning a timer to a job. Picking TIM6 for PWM is a bug you cannot fix in software.

A TIM has two parts: a **time-base unit** built around the counter, and multiple **channels** that use the counter's value to measure input signals or generate output signals. Channels are next week. This week is the time base.

And recall Week 03: enable the peripheral clock first. `RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;`

5.1 The signal chain

clock source → **prescaler** (PSC) → **counter** (CNT) → compare against ARR → overflow actions

Clock source. Several options: the internal clock `CK_INT` derived from `PCLK`, an external input pin, an external trigger, or internal triggers from another timer. We use the internal clock, selected by clearing `SMS` in `TIMx_SMCR` to zero.

Prescaler. This is the piece SysTick lacked.

The prescaler

A hardware divider between the clock source and the counter. It divides the input frequency by `PSC + 1`, where `PSC` is the 16-bit field in `TIMx_PSC`, so the division factor ranges from **1 to 65536**.

To prescale by 8, load `TIMx_PSC` with 7. Same +1 as SysTick's `RELOAD`, same reason.

Why it matters: the prescaler and the counter multiply. A 16-bit prescaler and a 16-bit counter give a combined division range up to $65536 \times 65536 \approx 4.3$ billion, which comfortably beats SysTick's 24 bits. **The prescaler buys range; the counter buys**

resolution. You choose how to split them.

Counter. TIMx_CNT, 16-bit or 32-bit, and it can count **up**, **down**, or **up/down** (centre-aligned). In up-counting mode it starts at zero and increments on each prescaler pulse.

Auto-reload. When CNT counts past the limit in TIMx_ARR, a hardware comparator detects the overflow and performs one or more actions:

- Clear the counter to zero.
- Set the update flag UIF in TIMx_SR, unless updates are disabled (UDIS = 1).
- Generate a TIMx interrupt, unless the update interrupt is disabled (UIE in TIMx_DIER is 0).
- Disable the counter if **one-pulse mode** (OPM = 1), giving exactly one count sequence rather than a repeating one.

6 Deep dive 5: The frequency formula

Here is the calculation. Learn both forms.

Timer overflow frequency

Up-counting or down-counting mode:

$$f_{\text{overflow}} = \frac{f_{\text{clock}}}{(\text{PSC} + 1)(\text{ARR} + 1)}$$

Up/down (centre-aligned) mode:

$$f_{\text{overflow}} = \frac{f_{\text{clock}}}{(\text{PSC} + 1) \cdot 2 \cdot \text{ARR}}$$

Note the centre-aligned form has a factor of 2, because the counter travels up and back down, and note it uses ARR rather than ARR + 1.

Worked, from the book. $f_{\text{clock}} = 48$ MHz, and we want a 100 kHz interrupt, using a prescaler of 32.

Prescaled clock: $48 \text{ MHz}/32 = 1.5 \text{ MHz}$. Then $\text{ARR} + 1 = 1.5 \text{ MHz}/100 \text{ kHz} = 15$, so $\text{ARR} = 14$ and $\text{PSC} = 31$.

Beware the trap

There is usually more than one correct answer, and they are not equally good.

For 100 kHz from 48 MHz you need a total division of 480. You could use $\text{PSC} + 1 = 1$ and $\text{ARR} + 1 = 480$, or $\text{PSC} + 1 = 32$ and $\text{ARR} + 1 = 15$, or $\text{PSC} + 1 = 480$ and $\text{ARR} + 1 = 1$. All give exactly 100 kHz.

They differ in **resolution**, and it matters enormously for PWM next week. The counter's range is your PWM resolution: with $\text{ARR} = 479$ you have 480 distinct duty-cycle steps, and with $\text{ARR} = 14$ you have 15.

Rule: prescale as little as you can get away with. Use the smallest prescaler that keeps ARR inside 16 bits, so the counter does as much of the division as possible. This is the single most useful piece of practical timer judgement in the course, and it is the reason a naive CubeMX configuration sometimes gives visibly steppy PWM.

In the lab

Your Lab 04 asks you to blink an LED at a chosen rate using TIM2 and to compare polling against interrupts. This formula is what you use to pick PSC and ARR, and it is worth doing by hand once even though CubeMX will compute it for you.

Worked for your board. Your project runs at 48 MHz. Say you want a 1 kHz control-loop tick, which is a sensible rate for a balancing robot. Total division needed is 48,000. Choose $PSC = 47$, so the prescaled clock is exactly 1 MHz and each count is exactly 1 μs , then $ARR = 999$ gives 1000 counts, so 1 ms.

That choice is deliberate rather than arbitrary. **Prescaling to a round 1 MHz makes every subsequent calculation trivial**, because ARR is then simply the period in microseconds. Anyone reading your code can check the timing in their head. That readability is worth real money when you are debugging a control loop at 2 a.m.

7 Deep dive 6: Periodic interrupts, with real numbers

Now put it together. Recall from Week 11's DAC discussion a triangle-wave generator written the naive way:

Listing 2: Book Listing 7.7. Correct, and it owns the entire CPU.

```

1 void Triangle_Output(void) {
2     int i = 0, change = 1;
3     while (1) {
4         DAC->DHR12R1 = i;
5         i += change;
6         if (i <= 0)                change = 1;
7         else if (i >= MAX_DAC_CODE - 1) change = -1;
8     }
9 }
```

This is Week 04's starter program wearing a different hat. The output rate is whatever the loop happens to run at, it is not adjustable, and no other code can ever execute.

The timer version moves the body into an ISR:

Listing 3: Book Listings 7.8 and 7.9. Note the debug pin trick.

```

1 void TIM3_IRQHandler(void) {
2     static int change = STEP_SIZE;
3     static uint16_t out_data = 0;
4
5     GPIOA->BSRR = DEBUG_ON_MSK;          /* scope marker: ISR entered */
6     TIM3->SR = ~TIM_SR_UIF;              /* clear the update flag */
7
8     out_data += change;
9     DAC->DHR12R1 = out_data;              /* output updated FIRST */
10    if (out_data < STEP_SIZE)
11        change = STEP_SIZE;
12    else if (out_data >= DAC_RESOLUTION - 1 - STEP_SIZE)
13        change = -STEP_SIZE;
14
15    GPIOA->BSRR = DEBUG_OFF_MSK;           /* scope marker: ISR exiting */
16 }
17
18 void Init_TIM3(void) {
19     TIM3->CR2 = 0;
20     TIM3->EGR = 0;
21     TIM3->PSC = TIM_PRESCALER - 1;        /* the -1 again */
22     TIM3->DIER = TIM_DIER_UIE;           /* enable update interrupt */
23     NVIC_SetPriority(TIM3_IRQn, 128);
```

```

24     NVIC_ClearPendingIRQ(TIM3_IRQn);
25     NVIC_EnableIRQ(TIM3_IRQn);
26     TIM3->CR1 |= TIM_CR1_CEN;           /* enable counter, last */
27 }

```

Recall the Week 07 NVIC ordering: priority, clear pending, enable. And note the counter is enabled *last*, after everything else is configured.

7.1 The measured cost

The book put a scope on that debug pin, and the numbers are the justification for the entire chapter.

Quantity	Measured
Interrupt period	10 μ s (100 kHz)
ISR execution time	$\approx 1.6 \mu$ s
Entry and exit overhead	$\approx 0.4 \mu$ s
Total CPU time per interrupt	$\approx 2 \mu$ s
CPU utilisation	2/10 = 20%
CPU left for other work	80%

Compare against the busy-wait version's 100%. Recall the CPU overhead definition from Week 04: that is a reduction from essentially all of the CPU to one fifth of it, for identical output.

The debug-pin technique, which you should steal

Set a GPIO pin high on entering an ISR and low on leaving it, then watch it on a scope or logic analyser. You get, directly and with no instrumentation overhead worth mentioning, the ISR's execution time, its frequency, and its jitter.

Using the scope's **infinite persistence** mode overlays every execution, so variation shows up as a smeared edge. The book measured 1.59 μ s typically, occasionally 1.7 μ s and occasionally 1.44 μ s.

This costs two BSRR writes, which recall from Week 03 are single atomic stores. It is the cheapest and most informative debugging tool in embedded work, and it tells you things a debugger cannot, because a breakpoint changes the timing it is trying to measure.

Beware the trap

Look at why the execution time varies, and then at why it does not matter here.

The variation comes from the conditional code testing whether `out_data` hit a bound. Different paths, different lengths.

But the DAC is written **before** that conditional. So the output is updated at a fixed offset from the interrupt every single time, and all the jitter lands *after* the thing that mattered. Timing accuracy is preserved.

That is a deliberate design choice and a genuinely good habit: **in a timing-critical ISR, do the time-critical output first and the bookkeeping afterwards**. Reorder those two blocks and the output jitters by 0.26 μ s for no reason at all.

Note also `TIM3->SR = ~TIM_SR_UIF`; at the top. Recall Week 07's `EXTI->PR`: the same write-to-clear discipline applies, and **failing to clear the flag makes the interrupt fire forever**. The

TIMx_SR register holds flags for the update event (UIF), the trigger (TIF), and each channel (CCyIF, CCyOF), grouped in one register so an ISR can identify the cause with a single read.

In the lab

Your Lab 04 has you do exactly this comparison, polling versus interrupt, and the manual gives you the HAL versions:

```

1 HAL_TIM_Base_Start(&htim2);           /* polling mode */
2 HAL_TIM_Base_Start_IT(&htim2);        /* interrupt mode */
3
4 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
5     if (htim->Instance == TIM2) {
6         HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_9);
7     }
8 }
```

Note the `if (htim->Instance == TIM2)` guard. Just like `HAL_GPIO_EXTI_Callback` in Week 08, this callback is shared by *every* timer in your program, so the guard is how you demultiplex. Omit it and enabling a second timer silently makes your LED toggle at the wrong rate.

Also note that `HAL_TIM_Base_Start_IT` clears and enables the interrupt for you, which is convenient and is why the register-level ordering above is worth understanding anyway: when it does not work, you need to know what it was supposed to have done.

8 Tying it back

We asked how to make something happen at an exact time without the CPU watching the clock, and the answer is a counter fed by a clock of known frequency, which converts pulses into time by multiplication, and which is immune to everything the software is doing.

SysTick is the small version, built into the core so every Cortex-M has one, with a 24-bit down-counter, a reload value, and the crucial $+1$: to divide by N you store $N - 1$. Its narrow range forced the 1 Hz example onto the divided-by-8 clock, and that limitation is exactly what motivates the TIM module's **prescaler**, which divides by $PSC + 1$ before the counter and thereby separates range from resolution. The overflow frequency is $f_{clock}/((PSC + 1)(ARR + 1))$, many combinations give the same frequency, and you should prescale as little as possible because the counter's range is your resolution.

Wire the overflow to an interrupt and you get a periodic tick whose cost is measurable and small: the book's 100 kHz waveform generator used 20% of the CPU where the busy-wait version used all of it, and a two-instruction debug pin on a scope is how you find that out.

Next week the channels. Each timer has several, each of which can either **capture** the counter when an input edge arrives, which is how you measure frequency and pulse width, or **compare** the counter against a threshold to drive an output, which is how you generate PWM. Plus the watchdog, which is the timer whose job is to notice that your program has stopped working.

Cheat sheet: Week 12

The core idea

A **counter** counts input pulses. Feed it an **unknown** source $\Rightarrow$ counts *events*. Feed it a source of **known frequency** $\Rightarrow$ measures *time*. Same circuit.

$t = \text{count} \times T_{\text{clock}}$. At 10 MHz, one count = $0.1 \mu\text{s}$, so a count of 15821 means $1582.1 \mu\text{s}$.

Unlike a busy-wait, the duration depends only on the input clock — not the compiler, optimisation level, wait states, or interrupts.

Five uses

1. **Periodic tick** — steady because hardware, not software, controls it.
2. **Timestamps** — extend range by counting overflow interrupts in a volatile software counter.
3. **Watchdog** — resets the MCU if not serviced.
4. **Frequency/period measurement** — **count events for fast signals** (± 1 on a big count is a small error), **measure period for slow signals**.
5. **PWM** — encode information as **duty cycle**, the fraction of time asserted. Digital, so noise-resistant. This is the fix for the Week 01 hot plate.

SysTick

In the Cortex-M core itself, so every Cortex-M has one with identical registers. **24-bit, counts down.**

SysTick- $\rightarrow$ VAL current (**writing anything clears it**) · $\rightarrow$ LOAD reload · $\rightarrow$ CTRL control.

CLKSOURCE 1 = processor clock, 0 = reference (**processor clock $\div$ 8**) · TICKINT enables the exception · ENABLE starts it · COUNTFLAG set on reaching zero, **cleared by reading CTRL**.

Divides by RELOAD + 1, so to divide by N store $N - 1$. (The sequence includes zero.)

Worked: 1 Hz from 48 MHz needs $N = 48,000,000$, but RELOAD maxes at $2^{24} - 1 = 16,777,215$. **Does not fit.** Use the $\div 8$ source: $N = 6,000,000$, RELOAD = 5,999,999. Fits. Max period ≈ 350 ms at 48 MHz, ≈ 2.8 s on the $\div 8$ clock.

On your board HAL_Init sets a 1 ms tick; HAL_Delay counts it. **Calling HAL_Delay in an ISR at or above SysTick's priority hangs forever.**

TIM time base

clock $\rightarrow$ **prescaler** PSC $\rightarrow$ **counter** CNT $\rightarrow$ compare with ARR $\rightarrow$ overflow actions.

Enable the clock first: `RCC->APB1ENR |= RCC_APB1ENR_TIM3EN;`

Internal clock selected by clearing SMS in TIMx_SMCR.

Prescaler divides by PSC + 1, range 1 to 65536. To divide by 8, load 7.

Counter counts **up**, **down**, or **up/down** (centre-aligned).

On overflow: clear CNT, set UIF (unless UDIS), interrupt (if UIE in TIMx_DIER), stop if **one-pulse mode** OPM.

Clear UIF in the ISR or it fires forever: `TIM3->SR = ~TIM_SR_UIF;`

Not every timer has every feature. Basic timers (TIM6/TIM7) have no channels — no PWM, no capture.

The formula. This is the exam calculation.

Up or down counting: $f_{overflow} = \frac{f_{clock}}{(PSC + 1)(ARR + 1)}$

Centre-aligned: $f_{overflow} = \frac{f_{clock}}{(PSC + 1) \cdot 2 \cdot ARR}$

Worked: 100 kHz from 48 MHz needs division by 480. With PSC= 31 ($\div 32 \Rightarrow 1.5$ MHz), ARR= 14.

Many combinations give the same frequency, and they are not equivalent. The counter range is your PWM resolution. **Prescale as little as possible;** use the smallest PSC that keeps ARR in 16 bits.

Useful trick: prescale to exactly 1 MHz (PSC= 47 at 48 MHz) and ARR becomes the period in μs .

Periodic interrupt cost, measured

100 kHz tick, ISR $1.6 \mu s$, entry/exit $0.4 \mu s \Rightarrow 2 \mu s$ of every $10 \mu s = 20\%$ CPU, 80% free. The busy-wait equivalent uses **100%**.

Debug-pin technique: BSRR high on ISR entry, low on exit, watch on a scope. Gives execution time, frequency, and jitter for two atomic stores. Infinite persistence shows the variation ($1.59 \mu s$ typical, 1.44 – $1.7 \mu s$ range).

Do the time-critical output first, bookkeeping after, so conditional-path jitter lands after the thing that mattered.

HAL: HAL_TIM_Base_Start_IT(&htim2), then HAL_TIM_PeriodElapsedCallback — **guard with** if (htim->Instance == TIM2), the callback is shared by every timer.

Likely exam prompts from this section

- Given f_{clock} and a desired frequency, compute PSC and ARR. *Near certain.*
- Why do PSC, ARR, and RELOAD all take $N - 1$ to divide by N ?
- Why is a hardware timer more accurate than a software delay loop?
- Why could the 1 Hz SysTick example not use the 48 MHz clock directly?
- Given several PSC/ARR pairs producing the same frequency, say which is best and why.
- When would you count events versus measure period to find a frequency?
- Compute CPU utilisation from an ISR execution time and an interrupt period.